

ENS 491-492 – Graduation Project

Draft Final Report

Project Title: Explainable AI-Powered Phishing URL Detector

Group Members:

Petek Metin - 32627

Ege Güray Ertan - 32648

Aliye Yolcu - 31942

İsmail Sarp Erdem - 32105

Arda Ösün - 32319

Supervisor(s): Dr. Orçun Çetin

Date: 10/05/2026



1. EXECUTIVE SUMMARY

Phishing websites are a constant security threat, designed to trick users into giving away passwords and other sensitive information. Most current detection tools depend on URL blacklists, IP reputation, or domain registration data, which only catch attacks that have been seen before and fail against new or short-lived pages. Recent machine learning methods improve accuracy but work as black boxes, with explainability added as a separate step rather than built in. This creates a gap between detection performance and the kind of trust needed in security-critical settings.

This project addresses that gap by building a content-based phishing detection system in which classification and explanation are produced jointly by a fine-tuned large language model. Instead of using external reputation services, the system analyzes structural and behavioral signals taken directly from webpages, such as login forms, brand mismatches, suspicious scripts, and unusual URL patterns. Detection is reformulated as a text generation task, so the model outputs a phishing or benign label together with a short natural-language reason in a single response.

The methodology centers on fine-tuning several open-source language models, using parameter-efficient training (LoRA) with 4-bit quantization on a single A100 GPU. A custom dataset built from multiple phishing and benign sources is used, with a categorized annotation that supervises explanation generation without being given to the model at inference time.

Early experiments showed that smaller models below two billion parameters were unstable, larger Llama variants produced fluent and signal-grounded explanations, and dataset bias initially inflated results. The dataset is currently being rebuilt for more reliable evaluation, which is ongoing.

In the later stages of the project, additional focus was placed on explanation grounding and reliability. The model was trained to generate structured outputs containing suspicious features, benign features, a rationale section, and a final verdict. Explanation supervision was derived from annotated phishing feature metadata during training, while these annotations were excluded at inference time to avoid data leakage. Besides classification accuracy, evaluation also included explanation-focused metrics such as grounding consistency, contradiction rate, and parse failure rate.

2. PROBLEM STATEMENT

Phishing is one of the most common types of online attacks. In a phishing attack, attackers create fake websites that look like real ones (such as banks, email services, or shopping sites) to trick users into giving away their passwords, credit card numbers, or other personal information. The number of phishing attacks keeps increasing every year, and most phishing pages are only online for a few hours before they are taken down or moved. This makes detection difficult, because by the time a phishing site is reported and added to a blacklist, many users have already been affected.

Most current phishing detection tools depend on URL blacklists, IP reputation databases, or domain registration information. These methods can only catch phishing pages that have already been seen before, so they often fail when the attacker uses a new domain or a short-lived page. More recent solutions use machine learning models to improve accuracy, but these models usually work like a black box: they make a decision but do not explain why. When explainability is added, it is often done later using separate tools, which makes the explanation feel disconnected from the actual model. This is a problem in security applications, where analysts need to understand and trust the system's decisions before acting on them.

The motivation for this project is to fix this gap by building a phishing detection system that focuses on the content and behavior of webpages and includes explainability from the start. Phishing pages are made to deceive users, and this goal usually leaves clear signs in the page itself, such as login forms that ask for sensitive information, mismatches between the brand shown on the page and the actual domain, suspicious scripts, missing privacy policies, and unusual URL patterns. By looking at these signs directly, the system can detect phishing even when there is no information about the domain's reputation or history.

This project also differs from earlier work in a few important ways. Many feature-based methods separate the classification step from the explanation step, adding explanation tools after training is finished. Other LLM-based phishing detectors focus mostly on accuracy and treat explanations as a side feature. In our project, we treat phishing detection as a text generation task, where a fine-tuned large language model gives both a label (phishing or benign) and a short reason for its decision in the same output. This way, the explanation is part of the model itself instead of being added later.

Another important challenge is that generated explanations may sound convincing even when they are not actually supported by the webpage signals. In security-related applications, an explanation that is fluent but incorrect can reduce analyst trust and make debugging more difficult. Because of this, the project focuses not only on prediction accuracy but also on producing explanations that are grounded in observable evidence from the webpage itself.

2.1. Objectives/Tasks

The project has the following objectives, with the expected result for each one:

1. **Define a list of phishing indicators based on webpage content and behavior.** *Expected result:* A grouped set of phishing signs covering domain-level signs (such as domain age and brand mismatch), HTML-level signs (such as login forms, hidden fields, and suspicious scripts), and URL-level signs (such as unusual characters or token patterns). This forms the base of the detection system.
2. **Build a clean and balanced dataset of phishing and benign pages.** *Expected result:* A dataset with enough variety and clear annotations of behavioral signs, ready to be used for both classification training and explanation training, without biases that would make the results look better than they really are.
3. **Reformulate phishing detection as a text generation task.** *Expected result:* A training data format where each webpage is shown as a text description of its signs, and the model is trained to output both a label and a short explanation that uses these signs.
4. **Fine-tune and compare different open-source large language models.** *Expected result:* A comparison of several model families and sizes (such as Llama, Qwen, DeepSeek, GLM) using parameter-efficient fine-tuning, in order to find the most reliable model under our compute limits.
5. **Use the metadata as a training signal instead of model input.** *Expected result:* A pipeline where the categorized phishing signs (the "families" field) are used to build the target explanation during training but are never given to the model at test time, so there is no data leakage but the explanations still match real phishing patterns.
6. **Add a RAG-based explanation layer to support the model's outputs.** *Expected result:* A retrieval module that connects the model's explanations to a small knowledge base of known phishing patterns, helping make the explanations more consistent and easier to check.
7. **Evaluate the system on both classification and explanation quality.** *Expected result:* An evaluation using standard metrics (Accuracy, Precision, Recall, F1, ROC-AUC), with Recall as the main metric because missing a phishing page is more harmful than a false alarm. A small qualitative check will also be done to see whether the explanations match the signs in the page and the model's prediction.

2.2. Realistic Constraints

Several real-world limits affected this project, and each one was handled during the work.

Compute limits. Fine-tuning large language models needs a lot of GPU memory and time, which we did not have at a large scale. Our main resource was a single NVIDIA A100 GPU through Google Colab Pro, with a limited number of compute units that sometimes ran out during experiments. Because of this, we did not retrain full models from scratch. Instead, we used Low-Rank Adaptation (LoRA), which only trains a small adapter and keeps the rest of the model frozen, reducing the number of trainable parameters to less than 1% of the full model. We also loaded the models with 4-bit quantization (BitsAndBytes NF4) to save memory. To compare

models without wasting compute, we used a step-by-step benchmarking plan: first running small experiments (1,000 samples per class), then increasing the size only for promising models.

Time limits. The project had to fit into the two-semester ENS491-492 schedule, so we had to limit our data collection and training scope. We collected data until the final experimentation and evaluation phase. If time allowed, we would have collected more data from more sources to increase training and evaluation quality. Time limits additionally led to us choosing pre-trained and smaller models.

Budget limits. Keeping massive amounts of large screenshots for a visual modal would be costly over the course of two semesters. Thus we had to limit the scope to text-only training and classification. This also caused feature extraction to be more difficult as non-text features were harder to detect, such as brand mismatches within a page.

Data quality limits. During the project, we found that our earlier feature selection and extraction led to potential biases. We redid the feature extraction with stricter checks and redesigned our fine-tuning methodology to overcome shortcut learning.

Explainability and feature representation limits. A challenge that became clear during the later stages of the project is that explainability depends not only on the model but also on the features given to it. The current set of behavioral features, while useful, is limited both in number and in how strongly each feature separates phishing from benign pages. Some features appear in both classes with similar frequency, which makes it harder for the model to use them as reliable evidence in its explanations. As a result, the model sometimes produces hallucinated explanations: outputs that sound reasonable but refer to signals that are not actually present in the page or that do not strongly support the prediction. This shows that explainability is not only a model-level problem but also a feature-engineering problem, and it cannot be fully solved by training alone. To address this, we are working on expanding the feature set with more discriminative signals and on tightening the link between training-time supervision and observable features, so that the model is encouraged to ground its explanations in evidence that is actually visible in the input.

Engineering and scientific standards. The system follows common practices for supervised learning with language models, including stratified train/validation/test splits, held-out evaluation, and reporting of standard classification metrics. The fine-tuning is built on well-known open-source libraries (Hugging Face Transformers, Unsloth, the SFT framework, and the LoRA/PEFT libraries). Confidence levels in the training data are based on how often each phishing sign actually appears in the dataset, instead of being chosen by hand.

3. METHODOLOGY

Describe in detail methods/techniques used in reaching the results. Depending on the project, methodology could include modelling, simulation, experiments, etc. You may have 2-5 pages in this section.

Data Collection

Due to the nature of the task at hand, we had to rely on human-labeled data. We decided to use OpenPhish for the phishing websites since they provide data for academic research purposes free of charge. We created a Docker container that would fetch the live phishing feed obtained from OpenPhish every 5 minutes and then scraping the websites via Scrapling. RDAP checks were made along with each website, and metadata such as redirects were kept as well. These websites were all labeled as phishing. Benign websites were obtained from Tranco top websites list that rank websites based on visit frequency. We took this as a source of truth due to the websites appearing in this list being trusted enough to be visited frequently. The same scraping was done on this list as well. Additionally, to get benign although suspicious websites, we collected data from [UrlScan.io](https://urlscan.io). Our source of truth for this website was VirusTotal and Google Safe Browsing scans. All of this data was compiled into a MongoDB database for easy access.

Data Analysis

After the earlier dataset issues described in Progress Report II, the team rebuilt the dataset and stored it in a single structured file where each row corresponds to one webpage. Each row carries three layers of information: the raw input (URL, page title, visible text, hostname, registered domain), a set of numerical measurements (URL length, form count, password input count, hidden input count, redirect count, external anchor ratio, and similar), and a list of detected signals with a label, a direction (suspicious, benign, or uncertain), a severity (low, medium, high), and a short human-readable statement. This structure was important for our work because the same row could be used both as model input and as a source for building grounded training explanations.

Before training, the dataset was inspected to understand its shape and check for problems that could bias the results. The first issue we found was that a small number of "mega-domains" had hundreds of rows each (for example, check-gaiastay.com with around 559 rows). If left untouched, these few domains would dominate training and test sets and make the model look better than it really was, because most test URLs would come from domains the model had already memorized. The second issue was duplicate URLs, which were removed before any further processing.

Feature Selection & Extraction

Because detection was framed as a text generation task, the feature representation given to the model is not a numerical vector but a short structured text description built from the dataset's three layers. For each URL, a builder function turns the row into a compact prompt that contains the URL and page title, a small set of page measurements (such as URL length, form count, credential form count, and external anchor ratio), and the list of detected signals. Each signal in the prompt is shown together with its direction and severity tag and its statement, so

the model sees not only that a signal fired but also how strong it is and whether it points toward phishing or benign.

The choice of which fields to include in this text was based on two ideas. First, only fields that were available consistently across the dataset were used, so the model would not learn to rely on signals that are only present in part of the data. Second, the fields that carry the clearest separation between phishing and benign pages (such as credential form presence, brand-domain mismatch, very young domains, and external anchor ratio) were prioritized in the prompt, while less informative measurements were dropped to keep the input length manageable.

To make these choices in a more rigorous way, the team also ran a separate feature analysis using machine learning. EBM was used alongside LIME and SHAP in order to validate features, alongside XGBoost together with SHAP. XGBoost was not used as the final classifier; it was used as a diagnostic tool that is well suited for measuring the contribution of each feature in a controlled way. The full feature set contained 36 features and the XGBoost baseline reached an F1 of 0.972 on a balanced 80,000-row sample, which gave a stable reference point for the analysis. Three analyses were then carried out on top of this baseline. The first was a leave-one-feature-out (LOFO) ablation, which trains one XGBoost model per feature with that feature removed and reports the drop in F1, giving a per-feature view of how much performance is lost without it. The second was a per-feature source check, where a single feature was used on its own to predict the *source* of the row rather than its label, in order to flag features that might be acting as artifacts of where the data came from rather than as real phishing signals. The third was SHAP global importance, computed on top of an XGBoost model trained on the full feature set, which gives a ranking of features by their average contribution to the model's decisions. Each feature was then labeled "keep" or "drop" based on the combination of the three results: features with no measurable impact in LOFO and SHAP were dropped as dead weight, features that could predict the source on their own were dropped as artifacts, and the rest were kept. Out of 36 features, 23 were kept and 13 were dropped before the final model training run.

Prompt Construction and Structured Outputs

The final prompt format was designed to produce structured and parseable outputs instead of free-form responses. Each training example asked the model to generate four sections: suspicious features, benign features, rationale, and final verdict. This format was chosen to make the explanations easier to evaluate and more consistent across predictions.

The suspicious and benign feature sections contain short feature identifiers linked to the signals detected from the webpage, while the rationale section explains the decision in natural language. The final verdict is generated as either "phishing" or "benign." By enforcing this structure during training, the model learns to organize its reasoning in a more controlled and interpretable way rather than producing unconstrained text outputs.

Leakage Control

Special care was taken to prevent data leakage between training and inference. During training, explanation supervision was generated using the annotated feature metadata available in the dataset. However, these annotations were never directly provided to the model as part of the inference input. Instead, the model only received the webpage-derived signals and textual summaries extracted from the page itself. This design ensured that the model learned to associate observable webpage behavior with phishing patterns, rather than memorizing annotation labels.

Model Experimentation

The model experimentation was split among the team across several model families. This subsection covers the experiments carried out on the Llama family of models (Llama-3.2-3B-Instruct and Llama-3.1-8B-Instruct), which were chosen because earlier exploration in Progress Report II had shown that Llama variants produced the most stable and signal-grounded outputs in our setting.

Both models were fine-tuned using Low-Rank Adaptation (LoRA) on top of 4-bit quantized base weights, which kept the memory cost low enough to run on a single A100 GPU. The LoRA configuration applied adapters to the attention and MLP projection layers (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj) with rank 16 and alpha 32, leaving less than one percent of the total parameters trainable. Training used the SFT framework from the Hugging Face TRL library with the adamw_8bit optimizer and bf16 mixed precision.

The training data followed a domain-aware split designed to prevent leakage between sets. Each registered domain was capped at 30 rows before splitting, so that a small number of mega-domains could not dominate the training budget with near-duplicate pages. After capping, the data was split so that all URLs from a given registered domain appeared in exactly one of the three sets, balanced between phishing and benign.

To choose how long to train each model, an early stopping test was run before the production training. In this test, both models were trained with a generous epoch budget while the validation loss was monitored after every epoch using a patience of 3, so that training would stop once the validation loss had not improved for three consecutive epochs. This way the model itself, rather than a guessed number, decided when learning had plateaued. For the Llama-3.2-3B model, the validation loss decreased steadily for the first few epochs and then started to flatten around epoch 7, while the loss curve for the Llama-3.1-8B model continued to improve a bit longer before reaching a similar plateau. Based on these observed plateaus, the production runs used fixed epoch counts of 3 epochs for the 3B model and 5 epochs for the 8B model, which sit safely inside the region where both models were still learning but had not yet

started to overfit. The validation set was kept in the production runs as well, but only for monitoring the loss curve during training, not for early stopping control.

A baseline evaluation of each model was run before any fine-tuning, using the same prompt format and the same test set. This served two purposes: it gave a clear "before" point for measuring the effect of fine-tuning, and it acted as a smoke test for the evaluation pipeline so that any bugs would surface before GPU time was spent on training.

The evaluation itself measures both classification quality and explanation quality. Classification is scored with accuracy, precision, recall, and F1, with recall on the phishing class treated as the most important metric, since missing a phishing page is more harmful in practice than raising a false alarm on a benign one. Parse failures (cases where the model's output does not contain a recognizable verdict) are also tracked, because a usable system needs to produce structured output reliably, not only correct labels. Explanation quality is measured with two grounding metrics: the *grounding ratio*, which is the fraction of the input's signal IDs whose key terms actually appear in the model's reasoning, and the *unsupported concept rate*, which is the fraction of predictions where the reasoning mentions a phishing-related concept (such as "iframe" or "certificate") that is not present in the input. The second metric is treated as a direct hallucination indicator, since it captures cases where the model is making up evidence that the page does not actually show.

Explanation quality is evaluated separately using grounding-based and consistency-based metrics. Feature grounding measures whether the model's explanation refers to signals that are actually present in the input webpage. The unsupported concept rate is used as a hallucination indicator, since it captures cases where the explanation mentions phishing-related concepts such as iframes, certificates, or suspicious forms even though those signals are not present in the input. Contradiction checks are also used to identify cases where the explanation conflicts with the final verdict. This allows the evaluation to measure not only whether the model predicts the correct label, but also whether its explanation is usable and faithful to the observed webpage evidence.

After the feature-level analysis described in the previous subsection, a final training and evaluation run was carried out on the cleaned feature set, with both Llama-3.2-3B-Instruct and Llama-3.1-8B-Instruct trained under the fixed-epoch plan and evaluated on a held-out test set of 1,500 URLs (750 phishing, 750 benign). For the Llama-3.2-3B model, the fine-tuned version reached an accuracy of 0.975 and an F1 of 0.975, with all 1,500 test samples producing parseable output and a faithfulness score (precision, recall, and F1 between the cited signals in the explanation and the signals actually present in the input) of 0.714 / 0.639 / 0.648. For the Llama-3.1-8B model, the fine-tuned version reached an accuracy of 0.987 and an F1 of 0.987, also with no parse failures, and faithfulness scores of 0.712 / 0.640 / 0.646, very close to the smaller model.

Two patterns are worth noting here. First, the 8B model gives a small but consistent classification gain over the 3B model (about one F1 point), while the faithfulness scores of the two models are almost identical, suggesting that the larger model is mostly better at deciding

the verdict rather than at grounding the explanation. Second, while the classification metrics are high, the faithfulness F1 of around 0.65 shows that the explanations only partially line up with the signals fired in the input, which means there is still meaningful room for the model to drift away from the actual evidence on some predictions.

User Facing Application

As a proof of concept, we created a user facing application that can be self hosted via Docker to use the model in a way that mimics real world usage. We created the application with Python Django that the user supplies the URL to scan. This sends the URL to a hardened Docker container with Scrapling that fetches the webpage, queries the domain and extracts metadata. The same features are then extracted from the raw data and sent back to the model, which classifies the site and provides an explanation.

4. RESULTS & DISCUSSION

Our initial accuracy goals were not met to the fullest extent, classical machine learning approaches still work better for pure classification tasks. However, we were able to beat GPT 5.4 on a sample set of 1000. The model loses on accuracy but gains on explanation accuracy, which was one of the main reasons for this project's development. We initially hoped to create a raw evidence based explanation model but could not figure out a way to automate the training, which we had to do due to time and manpower constraints.

At this stage, the project is still in the evaluation and model selection phase, so the results should be interpreted as preliminary findings rather than final conclusions. In one current evaluation run, the model reached an accuracy of 93.5%, a precision of 0.932, a recall of 0.773, and an F1-score of 0.845. These numbers show that the model was relatively strong when it predicted a page as phishing, because the high precision means that many phishing predictions were correct. However, the lower recall shows that the model still missed a noticeable number of phishing pages. Since phishing detection is a security-critical task, this recall gap is important. A false positive may inconvenience a user, but a false negative can allow a harmful phishing page to pass as benign. Therefore, this model configuration is promising as an explainable support tool, but it still needs improvement before it can be considered reliable as a standalone phishing detector.

The difference between precision and recall suggests that this model configuration behaved somewhat conservatively when assigning the phishing label. It avoided labeling benign pages as phishing too often, but this came with the cost of missing some phishing examples. This pattern may be related to the overlap between real login pages and phishing pages. Many benign websites also contain password fields, redirects, and minimal login interfaces, so the model sometimes requires stronger evidence before predicting phishing. This shows that credential-related signals are useful, but they are not enough by themselves. They need to be

interpreted together with other signals such as brand-domain consistency, navigation structure, support links, domain behavior, and the general context of the webpage.

For explanation quality, the same evaluation run produced a combined feature-grounding F1 of 0.709. This suggests that the explanations were often connected to the signals present in the input, but the grounding was not perfect. The suspicious feature grounding score was 0.571, while the benign feature grounding score was 0.651. This means that the model was better at explaining benign evidence than citing the exact suspicious evidence behind phishing predictions. This is an important limitation because the project aims not only to classify webpages, but also to explain the decision in a way that is useful for human inspection.

Text-similarity metrics were also used as supporting indicators for explanation quality. In the same run, the model reached a ROUGE-L score of 0.692 and a BLEU score of 0.544 when compared with reference explanations generated from annotated feature metadata. These scores suggest that the generated rationales often followed the expected explanation style and vocabulary. However, these metrics alone are not enough to prove faithfulness. An explanation can sound similar to the reference explanation while still mentioning weak, generic, or unsupported evidence. For this reason, grounding and contradiction checks are treated as more important than ROUGE-L and BLEU when judging whether the explanation is actually reliable.

The contradiction rate was relatively low at 3%, which is a positive early sign. This means that in most cases, the rationale did not directly conflict with the final verdict. However, the parse failure rate was 6.5%, with 13 out of 200 outputs failing to follow the expected structured format. This is important because the system is designed to generate parseable sections such as suspicious features, benign features, rationale, and verdict. Even if a prediction is correct, an unstructured or incomplete output makes the system harder to use in a real application. Therefore, parse failure is not only a formatting issue, but also a practical reliability issue for generative classification systems.

Error analysis from the current experiments showed that many incorrect predictions were concentrated around a few recurring patterns rather than being completely random. One common issue involved hosted-platform impersonation pages using domains such as github.io, vercel.app, or pages.dev. These platforms are legitimate infrastructure, so the domain itself may not look suspicious at first. However, attackers can still use them to host fake login pages or brand impersonation pages. This makes detection harder because the model needs to separate the reputation of the hosting platform from the actual behavior and content of the webpage.

Another difficult group involved legitimate authentication and single sign-on pages. These pages often contain password fields, login buttons, redirects, and limited visible text, which are also common in phishing pages. Because of this overlap, the model sometimes treated real authentication flows as suspicious. This shows that credential-related signals are useful, but they are not enough by themselves. They need to be interpreted together with brand consistency, domain ownership, navigation structure, privacy or support pages, and other legitimacy indicators.

The explanations also sometimes relied too heavily on weaker generic signals, such as null anchor usage or iframe-related features, instead of stronger phishing-specific indicators. This contributed to cases where the explanation sounded reasonable but was not fully supported by the most important evidence present in the page. For example, a page may include a weak structural signal that appears in both phishing and benign pages, and the model may still use it as if it were strong evidence. This shows that explanation quality depends not only on the language model, but also on the quality and discriminative power of the extracted features.

Another important observation was that output formatting reliability became an evaluation concern by itself. Since the model is expected to generate a structured response format, outputs that fail to follow the expected sections reduce the usability of the system even if the classification itself is correct. This is different from traditional machine learning classifiers, where the output is usually a fixed label or probability. In this project, the output is both a prediction and an explanation, so the structure of the response becomes part of the system's reliability. Parse failure rates are therefore useful for measuring how consistently the model follows the required format.

The comparison with classical machine learning also clarified the trade-off in the project. Classical machine learning models, such as XGBoost, can perform very strongly on structured numerical features, especially when the features are clean and well-separated. However, classical models do not naturally produce human-readable explanations as part of the prediction. They require a separate explanation method, which may not always align naturally with the final user-facing output. The LLM-based approach has not yet fully outperformed classical models in raw classification performance, but it provides a structured explanation together with each decision. This makes it more suitable for analyst-facing scenarios, where understanding the reason behind a prediction is also important.

Compared with general-purpose LLM prompting, fine-tuning appears to provide a more controlled output format. The structured format helped the model produce suspicious features, benign features, a rationale, and a final verdict in a more consistent way. This is important because a cybersecurity tool cannot rely on long and inconsistent free-form explanations. However, the remaining parse failures show that structured generation is still not fully solved. More training examples, stricter formatting constraints, or post-processing rules may be needed to make the output reliable enough for real deployment.

Overall, the current results show useful progress toward explainable phishing detection, but they should not be treated as final proof of system performance. The model can generate understandable explanations and often connects them to webpage signals, but the explanations do not always perfectly match the actual evidence in the input. The classification results are promising, especially in terms of precision, but recall needs improvement because missed phishing pages are costly in real-world security settings. The explanation results also show that grounding is harder than generating fluent text. A fluent rationale is not enough; the explanation must also be faithful to the observed signals.

These current findings suggest three main directions for the next stage. First, LLM-based phishing detection can produce useful structured explanations, but output format reliability must be improved because parse failures reduce usability. Second, explanation faithfulness remains a challenge: the model can generate fluent rationales, but these rationales do not always perfectly match the actual signals in the input. Third, feature quality directly affects explanation quality. If the extracted signals are weak, missing, or ambiguous, the model has less reliable evidence to use in its explanation. Therefore, the next steps should focus on improving feature extraction, strengthening explanation grounding, tuning thresholds, and comparing model configurations before selecting the final system.

5. IMPACT

This project has potential impact in both cybersecurity research and practical phishing detection systems. Since the system is designed around small and open-source language models, it can be more accessible than approaches that depend on very large proprietary models or commercial security APIs. This makes the system easier to reproduce, modify, and deploy in academic or small-scale security settings. The use of smaller models also has an efficiency advantage, since they can potentially run on consumer-level or institutional hardware rather than requiring large datacenter-level infrastructure.

Another important impact of the project is its focus on explainability. In many phishing detection systems, the user only receives a final label such as “phishing” or “benign.” In this project, the model is designed to also provide a short explanation based on webpage signals. This can make the system more useful for analysts, students, or security teams who need to understand why a page is considered suspicious. Even when the model is not used as a fully automated detector, it can still support human decision-making by highlighting possible phishing indicators.

From a research perspective, the project also contributes by combining phishing detection with structured explanation generation. The datasets, feature extraction logic, and training notebooks are planned to be made open source, which would allow other researchers or students to reproduce the experiments and build on the work. This is especially useful because explainable phishing detection is still a challenging problem, and future work can improve the feature set, explanation grounding, and model reliability.

There are also legal and ethical considerations. The models used in the project are based on open-source or research-permitted licenses, and the phishing data sources were selected with academic research use in mind. However, scraping benign websites from sources such as the Tranco list may raise Terms of Service concerns depending on how the data is collected and used. To reduce possible harm, the scraping pipeline was designed to visit pages minimally and avoid unnecessary load on websites. Before any public release or real deployment, the licensing

conditions of the models, datasets, and collected webpage content should be reviewed carefully.

Overall, the main impact of the project is not only building another phishing classifier, but exploring how phishing detection can become more transparent, reproducible, and accessible. The system still has limitations, especially in recall and explanation faithfulness, but it provides a useful foundation for future work on explainable cybersecurity tools.

6. ETHICAL ISSUES

This project involves several ethical considerations, mainly related to data collection, website scraping, and the use of security-related datasets. Since the system requires webpage content for feature extraction, the team collected data from live websites. To reduce possible harm, the scraping pipeline was designed to visit each URL only once and avoid repeated requests to the same website. This helped minimize unnecessary traffic and reduced the chance of disrupting normal website operation.

Another ethical concern is the collection of benign website data. Even though the project does not collect personal user data, scraping public webpages can still raise concerns related to website terms of service and responsible data use. For this reason, the collected data was used only for academic research purposes, and the team avoided collecting sensitive user information such as account credentials, private form submissions, or personally identifiable information.

For phishing websites, the project used data sources intended for research or security analysis. However, phishing websites may contain malicious scripts, redirects, or deceptive content. To reduce risk, webpage fetching and analysis were performed inside an isolated container environment. This separation helped protect the local system and prevented direct interaction with potentially harmful pages.

There is also an ethical responsibility in how the model's predictions are presented. A phishing detection system can produce both false positives and false negatives. A false positive may incorrectly flag a legitimate website, while a false negative may allow a harmful phishing page to pass as benign. Because of this, the system should not be treated as a fully automated decision-maker at this stage. Instead, it is better understood as a support tool that provides a prediction and an explanation for human review.

Finally, the explainability component introduces its own ethical issue. If the model produces an explanation that sounds convincing but is not actually supported by the webpage evidence, it may mislead users or analysts. For this reason, the project includes explanation-grounding checks and treats explanation reliability as part of the evaluation, not just classification accuracy.

7. PROJECT MANAGEMENT

One of the main project management challenges was the dependency between data collection, feature extraction, and model training. Since the final dataset and feature extraction pipeline were not fully ready in the earlier stages, the team first experimented with precurated datasets to test the fine-tuning workflow and understand model behavior. Also, earlier datasets were not fit to use in the final product, so the earlier time spent collecting data was not utilized fully. Although these early experiments did not fully represent the final task, they were useful for identifying training issues, testing prompt formats, and learning which model families were more stable, in addition to leading us to collect more explainable data over time.

After the project moved closer to the final data format, it became clear that working with data that better matches the final system is important. Some early compute resources were spent on datasets that were later replaced, but this also helped the team discover problems earlier, such as model collapse, output formatting issues, and the need for more careful evaluation. From a project management perspective, this showed the importance of building a small end-to-end prototype as early as possible, even before the full dataset is complete.

Another important project management decision involved moving the later training runs from Google Colab Pro to the Tosun HPC cluster at Sabancı University. Earlier experiments on Colab were affected by limited compute units and session interruptions, which made long-running experiments difficult to reproduce reliably. Migrating to the HPC environment allowed longer uninterrupted training sessions, better experiment tracking, and more stable model evaluation.

8. CONCLUSION AND FUTURE WORK

Our dataset is still numerical, and small/large language models still struggle to work with numbers. This leads to accuracy loss over machine learning models grounded in mathematical structure. Future work could be the creation of more text based features or entirely evidence based feature extraction and fine tuning. A stronger dataset, reinforcement learning with human feedback and agent alignment training could be used to create a general explainability framework with evidence attributing, not constrained to phishing classification.

Additionally, our current dataset relies on a relatively narrow set of features. This low feature count limits the model's ability to capture the patterns often found in phishing websites. Focusing on feature engineering will improve the models' performance both in prediction and explainability.

Another important future direction is improving explanation grounding and reducing hallucinated reasoning. While the current system can generate fluent explanations, the explanations do not always perfectly align with the actual webpage signals. Future work could focus on stronger supervision methods, better feature engineering, and more robust grounding mechanisms to improve explanation reliability in security-critical settings.

